

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Benchmarking and Performance Analysis of LLM Inferencing Systems

Saksham Rathi

Systems for LLMs
Under Prof. Mythili Vutukuru

Indian Institute of Technology Bombay
Autumn 2025-26

Contents

1. Workload Picked
2. Batch of Requests through Various Engines
3. LLM Request Timelines
4. Throughput vs Load for vLLM
5. NSYS Measurements
6. GPU Memory Breakdown
7. Experiments with Multiple Families
8. Broader Goal

Workload Setup

- **Goal:** Evaluate inference performance of LLMs under short queries with large shared context.
- **Model:** Qwen 0.5B, Tiny Llama 1.1B, served using vLLM/SGLang/Transformers.
- **Hardware:** NVIDIA RTX6000 48GB GPU.
- **Metrics:** Throughput (req/s), latency, GPU utilization.

Parameter	Value/Range
System Prompt Length	≈1500 tokens (shared across requests)
Query Rate	5–700 req/sec
Query Length	10–20 tokens
Output Length	30–50 tokens
Batch Size	Adaptive

Table: Workload Characteristics

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Experiment Setup: Engine Comparison



Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

- Compare inference engines: **Transformers**, **vLLM**, **SGLang**, **NEO**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Batch of requests through various Engines

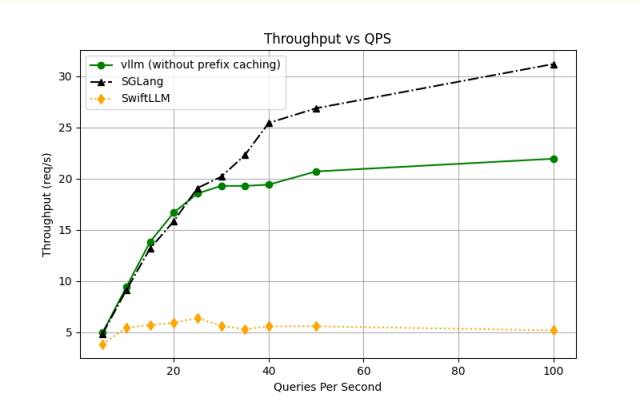


Figure: Throughput comparison across varying workloads

- **SwiftLLM:** lightweight vLLM clone (∼2000 LOC), but doesn't performs that well.
- **SGLang:** achieves best performance via radix-tree prefix sharing.

Technique Used	Time (s)
Transformers (batch = 1)	30.21
Transformers (batch = 2)	19.51
Transformers (batch = 3)	18.64
vLLM (no system prompt pre-filling)	0.45
SGLang (fork)	1.03
SGLang (run_batch)	0.40
NEO	0.55

Table: Measuring the time taken to process a batch of 24 requests using different techniques

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Experiment Setup: Request Timelines



Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

- Aim: Measure time spent in **prefill** vs **decode** stages for 20 req/sec workload (sending a new request after every 0.05 seconds).
- Using vLLM V0 engine without chunked prefill.
- Tool: Custom logging of per-request timestamps.

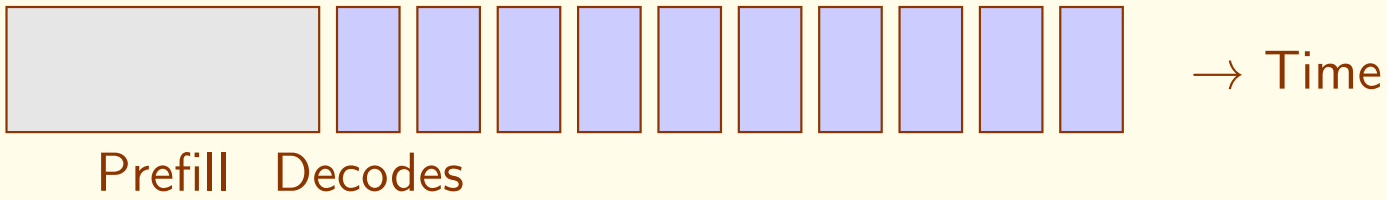


Figure: Simplified per-request timeline

Results: LLM Request Timelines

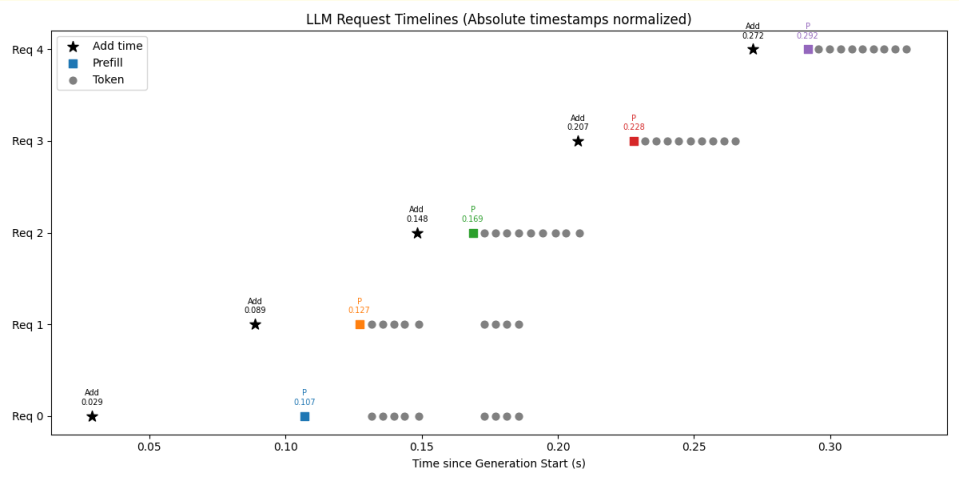


Figure: 20 Requests/sec – No chunked prefill – vLLM V0

- Prefill dominates runtime due to large prompt.
- Prefill and decode cannot overlap here.
- Low TTFT but high total latency.

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Results: Throughput vs Load for vLLM

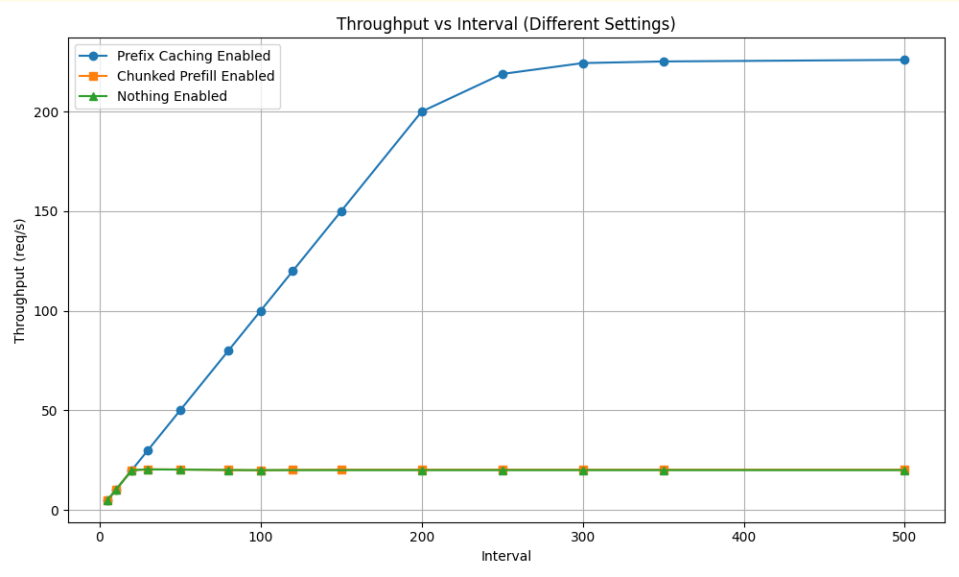


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Results: NSYS Measurements

Goal

Understand compute vs memory utilization and bottlenecks.



Figure: NSYS Metrics Snapshot

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Observation: NSYS Profiling



Systems for LLMs

Authors

Workload Picked

Batch of Requests through Various Engines

LLM Request Timelines

Throughput vs Load for vLLM

NSYS Measurements

GPU Memory Breakdown

Experiments with Multiple Families

Broader Goal

- Compute: SM Active and Warp occupancy.
- Memory: vLLM books full GPU memory upfront.
- vLLM scheduler stats show the actual KV Cache Usage.
- PCIe Bandwidth low $\rightarrow$ small workloads, minimal CPU transfer.

GPU Memory Breakdown

Component	Memory Occupied
Weights	0.93 GiB
Peak Activation	4.07 GiB
Non-Torch	0.02 GiB
CUDA Graph	0.41 GiB
KV Cache: Intersection (Prefix)	1,280 tokens / 0.0157 GiB
KV Cache: Extras for Requests	640 tokens (10 requests × 64 tokens) / 0.0078 GiB
KV Cache: Free	2,672,880 tokens / 32.8445 GiB

Table: GPU Memory Breakdown and KV Cache Usage

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Observation: Memory Profiling



Systems for LLMs

Authors

Workload Picked

Batch of Requests through Various Engines

LLM Request Timelines

Throughput vs Load for vLLM

NSYS Measurements

GPU Memory Breakdown

Experiments with Multiple Families

Broader Goal

- KV Cache footprint is tiny — prefix reuse highly memory-efficient.
- Most GPU memory remains free → headroom for larger batches.
- Suggests potential for multi-family prefix caching in future work.

Generation of these families



- Let's say we already have a large system prompt of length 1500 tokens.
- We generate multiple families of requests by prepending a unique family identifier to the system prompt.
- For example, for family 1, we prepend "AA", for family 2, "AB", and so on.
- This ensures that although the families have a large substring in common, but their prefixes always differ, and the KV Cache block hashes won't match.
- This in turn implies, that we need to compute and store separate KV Cache blocks for each family.

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Experiments with Multiple Families

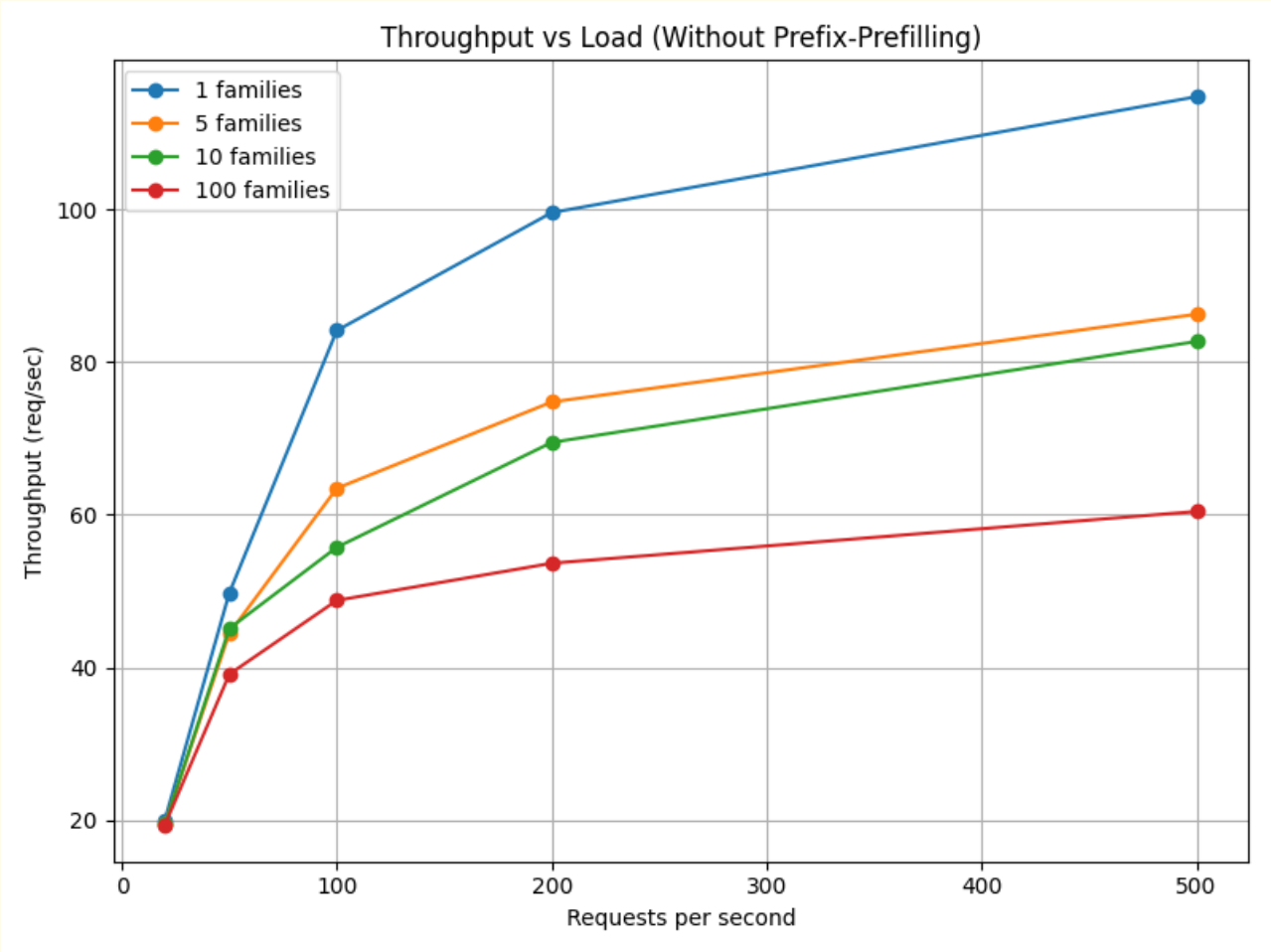


Figure: Throughput vs Load without prefix pre-filling

Systems for LLMs

Authors

Workload Picked

Batch of Requests through Various Engines

LLM Request Timelines

Throughput vs Load for vLLM

NSYS Measurements

GPU Memory Breakdown

Experiments with Multiple Families

Broader Goal

Experiments with Multiple Families

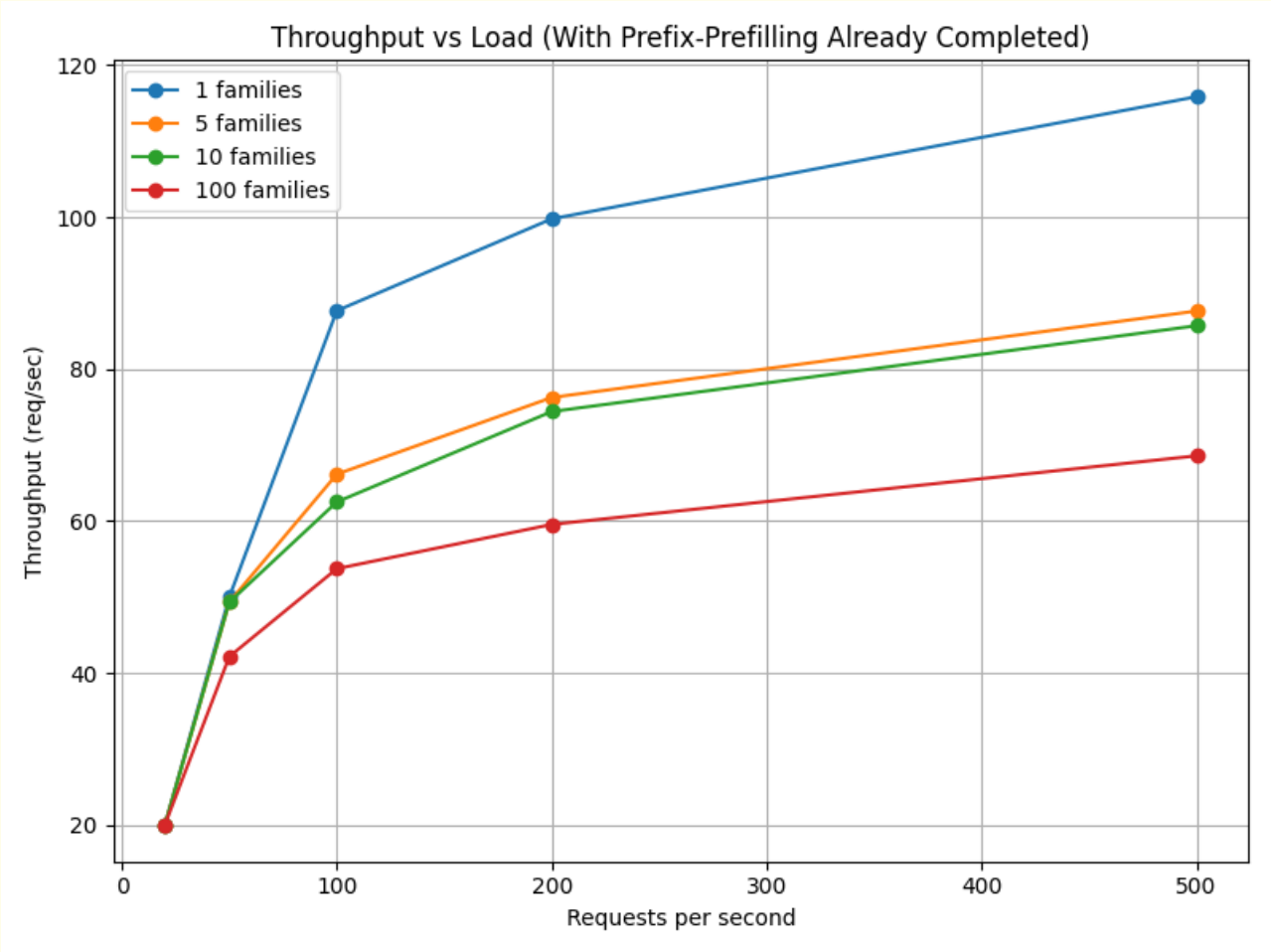


Figure: Throughput vs Load with prefix pre-filling

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Experiments with Multiple Families

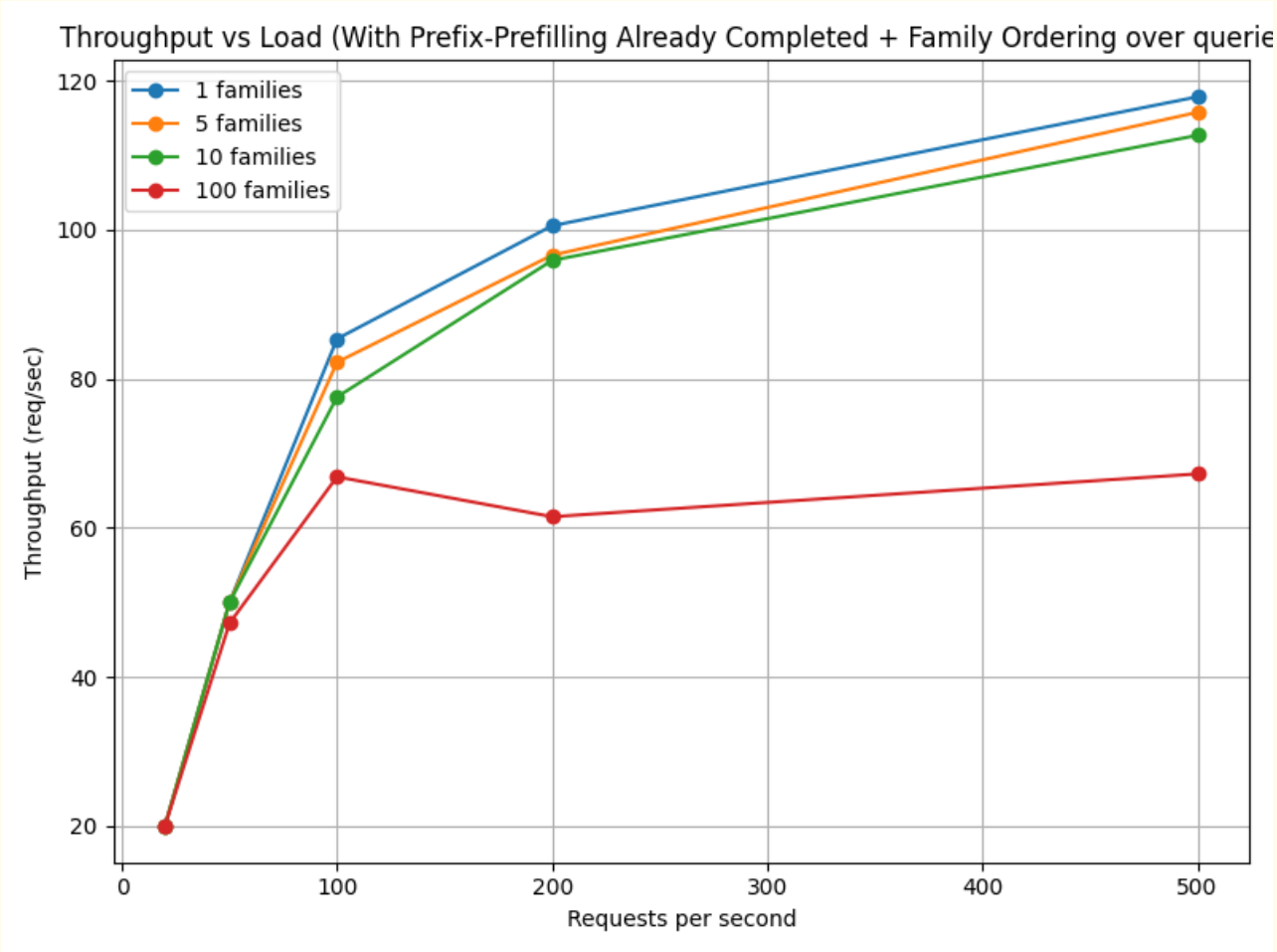


Figure: Throughput vs Load with prefix pre-filling with ordered family IDs

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Experiments with Multiple Families

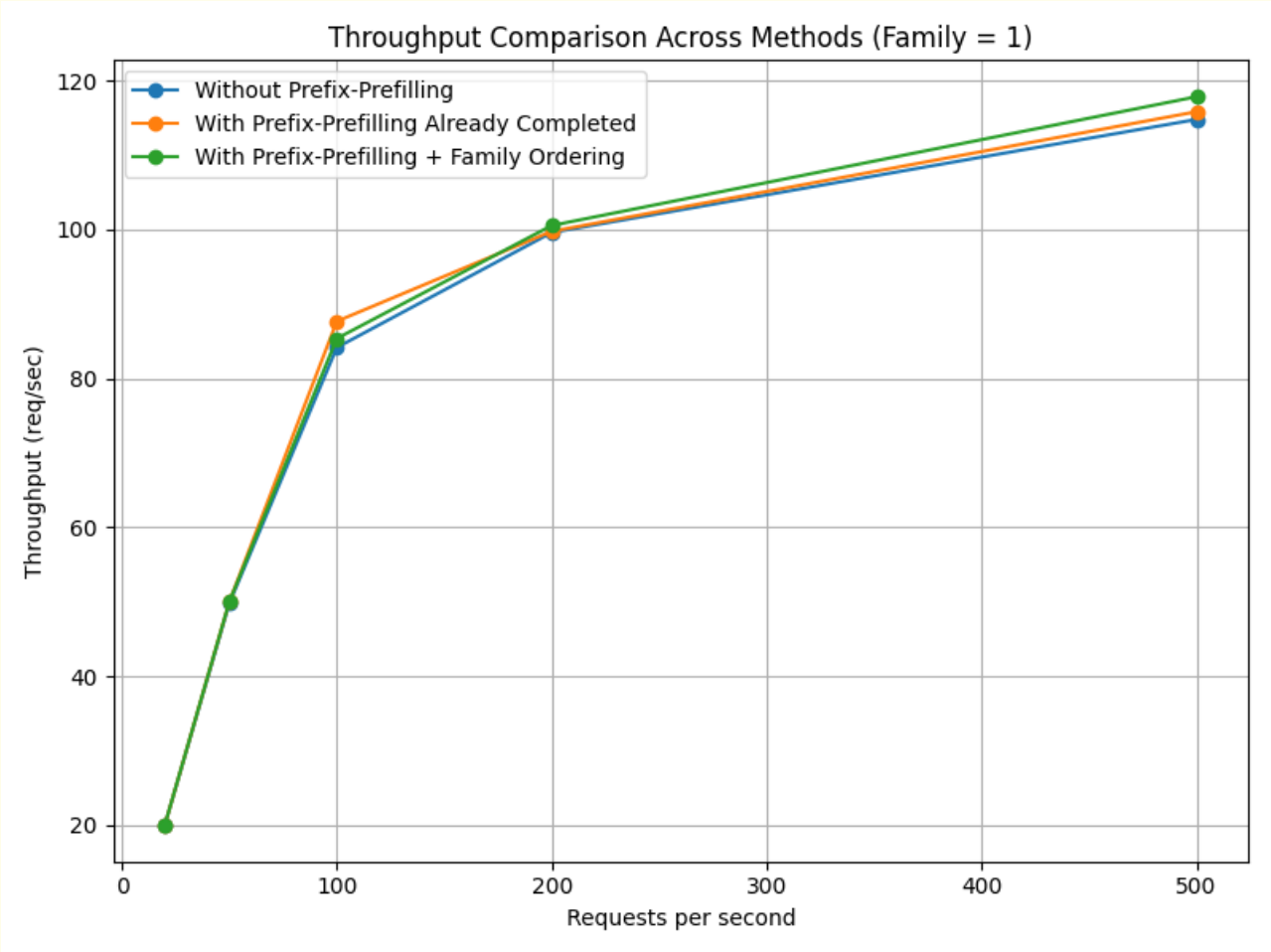


Figure: Throughput vs Load for 1 family

Systems for LLMs

Authors

Workload Picked

Batch of Requests through Various Engines

LLM Request Timelines

Throughput vs Load for vLLM

NSYS Measurements

GPU Memory Breakdown

Experiments with Multiple Families

Broader Goal

Experiments with Multiple Families

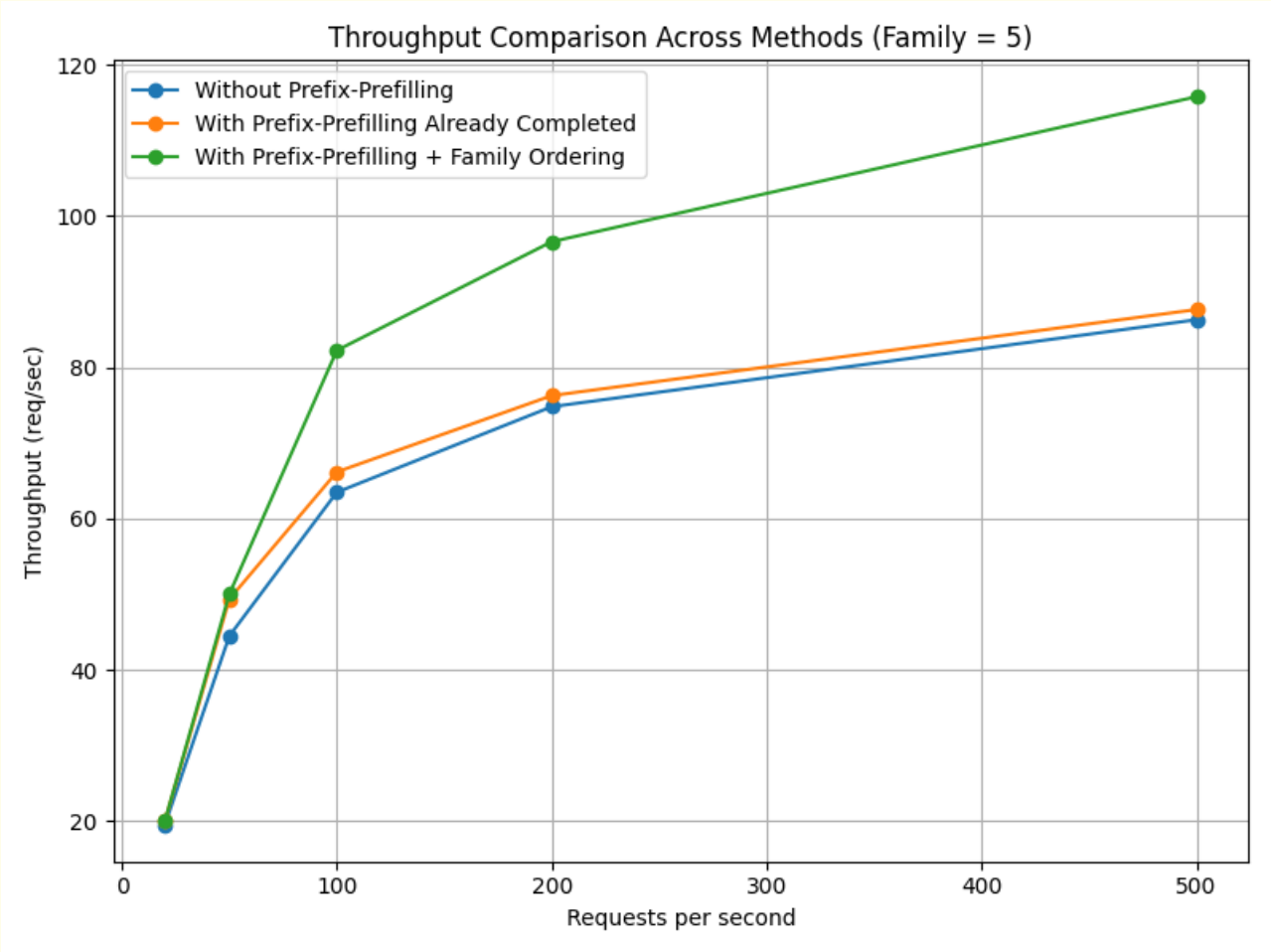


Figure: Throughput vs Load for 5 families

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Experiments with Multiple Families

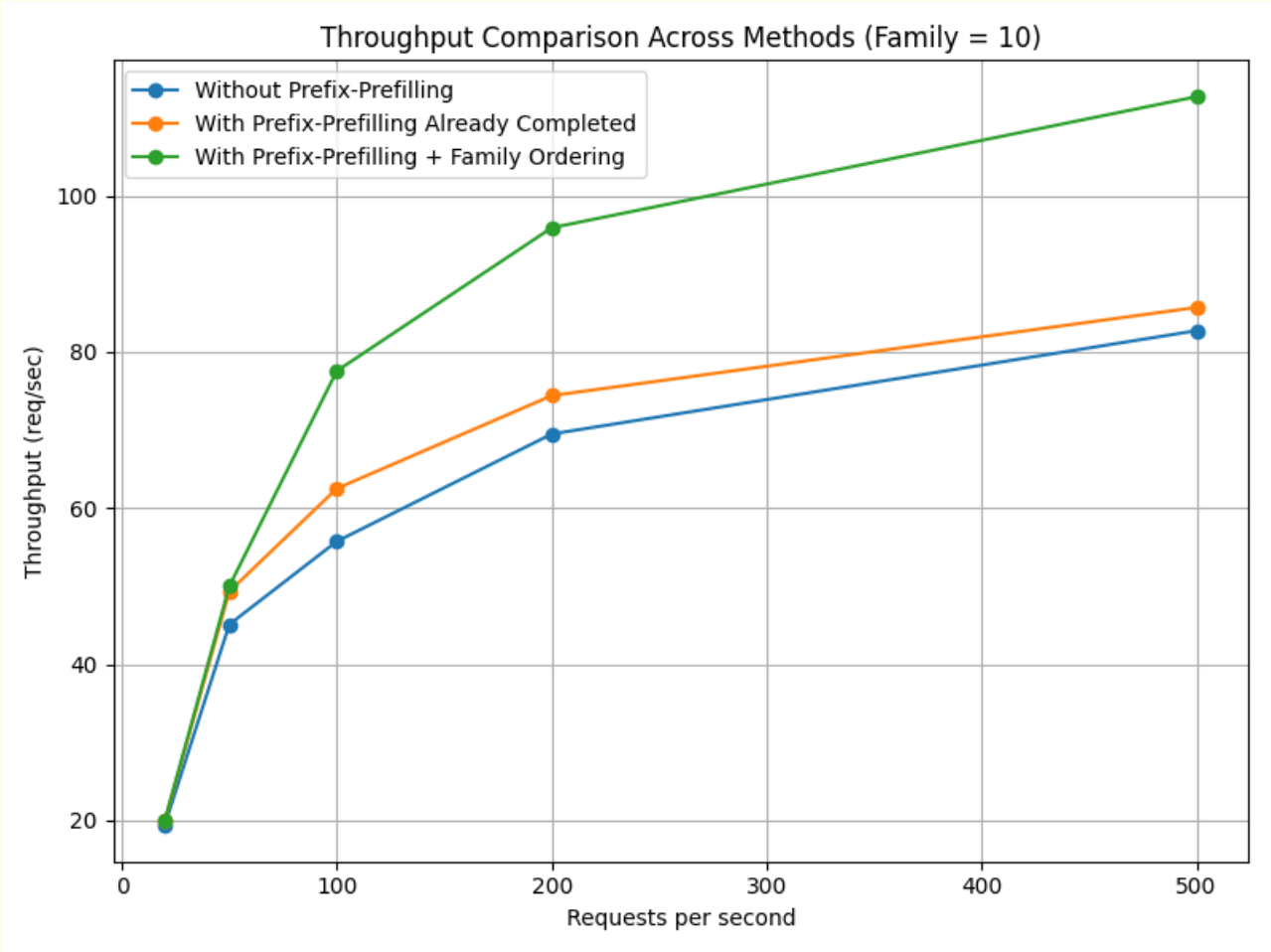


Figure: Throughput vs Load for 10 families

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Experiments with Multiple Families

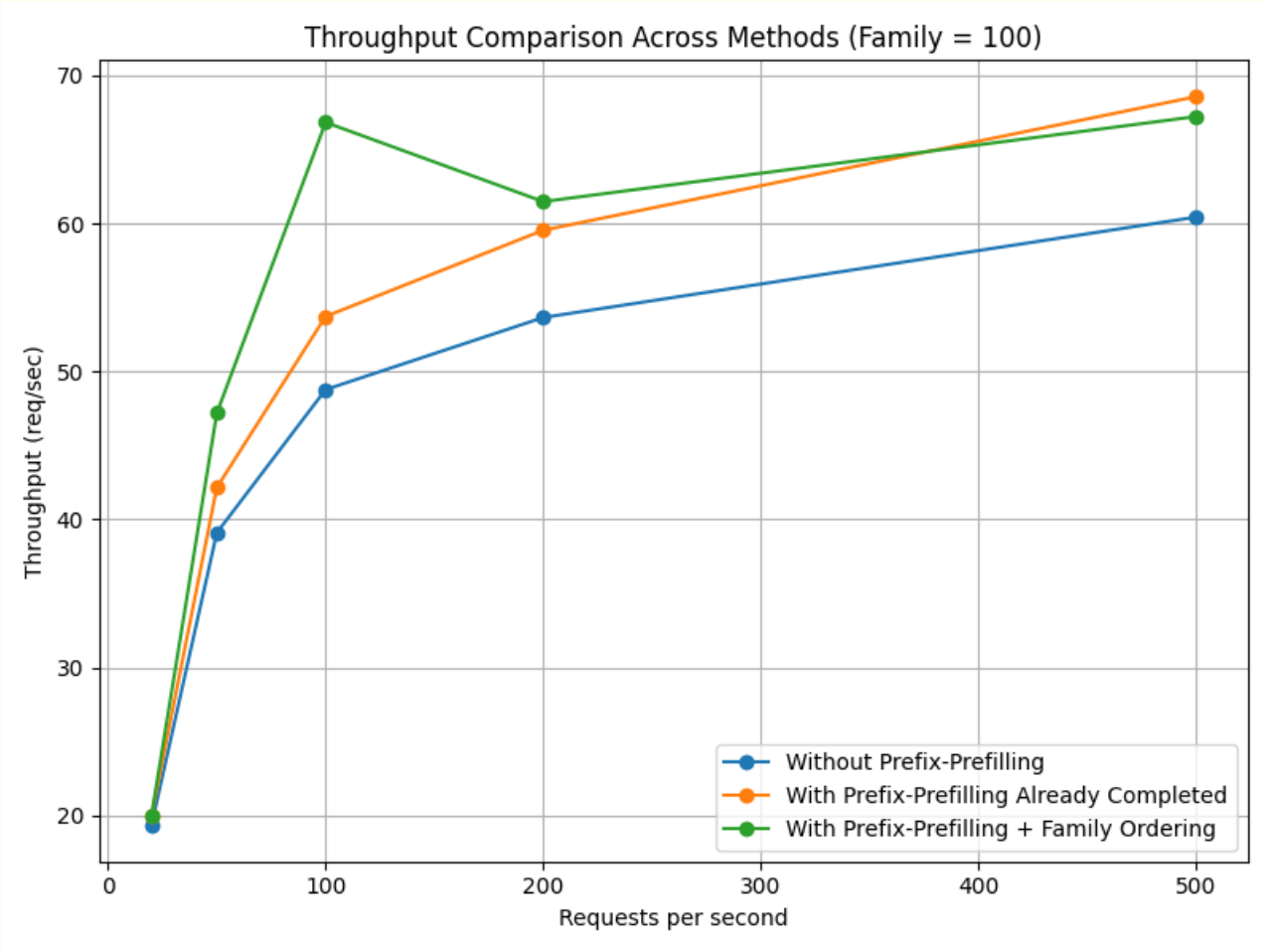


Figure: Throughput vs Load for 100 families

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Broader Goal: Motivation

- **Without prefix sharing:** batch limited by model context length (e.g., 32,768 tokens for a small Qwen model $\rightarrow$ 1500 tokens $\times$ 20 requests).
- **With prefix sharing:** utilization-limited, not context-limited. As the number of concurrent requests grows, per-request latency increases, and beyond a point, overall throughput would start declining.

Next Step

Design a scheduler for families of such requests. At each scheduling step, we must decide whether to prefill a shared prefix or decode tokens for an existing request.

- Explore KV cache offloading to CPU for extended workloads.

Systems for LLMs
Authors
Workload Picked
Batch of Requests through Various Engines
LLM Request Timelines
Throughput vs Load for vLLM
NSYS Measurements
GPU Memory Breakdown
Experiments with Multiple Families
Broader Goal

Future Directions: Quality vs Performance



- Strategy 1: Pass full system prompt each time.
- Strategy 2: Fine-tune for families of similar requests.
- Study trade-offs:
  - **Performance:** latency, throughput, memory use.
  - **Quality:** output accuracy, coherence.



Thank You